

XCS221 Assignment 3 — Route Planning

Due Sunday, April 12 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at <http://xcs221-scpd.slack.com/>
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset $\text{\LaTeX}$ submission. If you wish to typeset your submission and are new to $\text{\LaTeX}$, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Also note that your answers should be in order and clearly and correctly labeled to receive credit. Be sure to submit your final answers as a PDF and **tag all pages correctly when submitting to Gradescope**.

Coding Submission: Some questions in this assignment require a coding response. For these questions, you should submit only the `src/submission.py` file in the online student portal. For further details, see Writing Code and Running the Autograder below.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. More information regarding the Stanford honor code can be found at <https://communitystandards.stanford.edu/policies-and-guidance/honor-code>.

Writing Code and Running the Autograder

All your code should be entered into `src/submission.py`. When editing `src/submission.py`, please only make changes between the lines containing `### START_CODE_HERE ###` and `### END_CODE_HERE ###`. Do not make changes to files other than `src/submission.py`.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- **basic:** These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.
- **hidden:** These unit tests are the evaluated elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests. These evaluative hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the [Anaconda Setup for XCS Courses](#) to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Hidden Tests

All hidden tests rely on files that are not provided to students. Therefore, the tests can only be run remotely. When a hidden test like `1a-1-hidden` is executed locally, it will produce the following result:

```
----- START 1a-1-hidden: Test multiple instances of the same word in a sentence.
----- END 1a-1-hidden [took 0:00:00.011989 (max allowed 1 seconds), ???/3 points] (hidden test ungraded)
```

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic: Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic: Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None ← This error caused the test to fail.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a") ← In this case, start your debugging
                                              in line 23 of grader.py.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

Remote Execution

Basic and hidden tests are treated the same by the remote autograder. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError': {'a': 2, 'b': 1} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError': {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

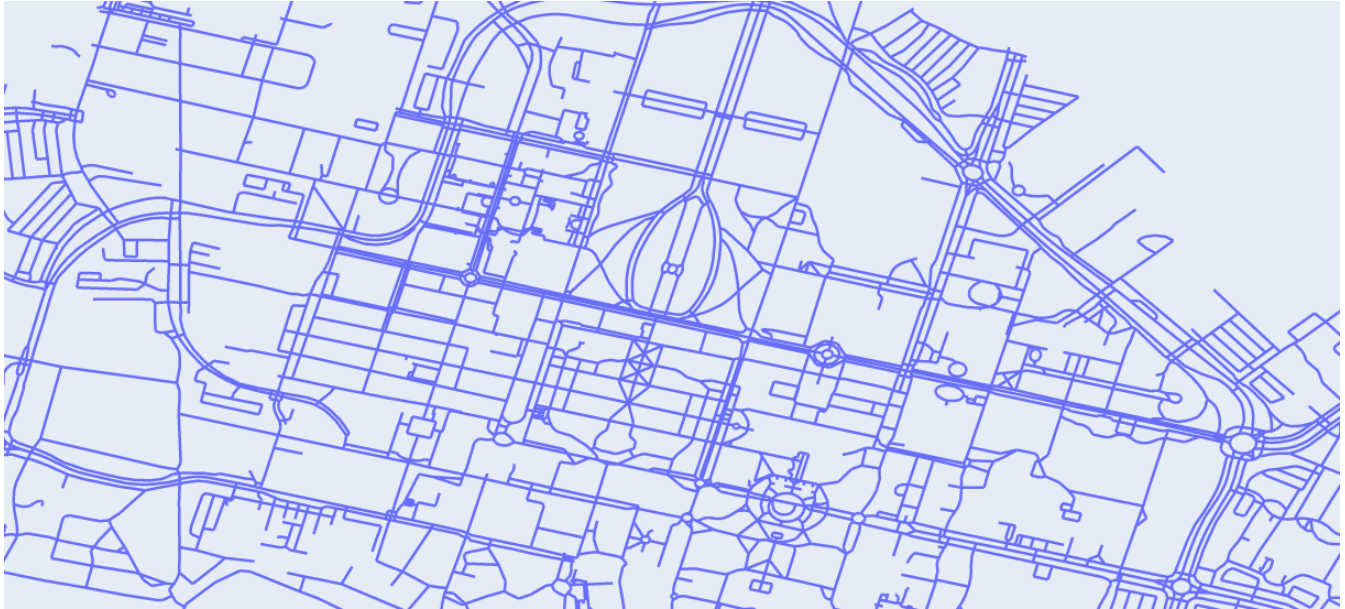
Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

Introduction



In route planning, the objective is to find the best way to get from point A to point B (think Google Maps). In this homework, we will build on top of the classic shortest path problem to allow for more powerful queries. For example, not only will you be able to explicitly ask for the shortest path from the Gates Building to Coupa Cafe at Green Library, but you can ask for the shortest path from Gates back to your dorm, stopping by the package center, gym, and the dining hall (in any order!) along the way.

We will assume that we have a *map* of a city (e.g., Stanford) consisting of a set of *locations*. Each location has:

1. a unique label (e.g., 6608996258),
2. a (latitude, longitude) pair specifying where the location is (e.g., 37.4299866, -122.175519), and
3. a set of *tags* which describes the type of location (e.g., amenity=food).

There are a set of *connections* between pairs of locations; each connection has a *distance* (in meters), and can be traversed in both directions (if the distance from A to B is 100 meters, then the distance from B to A is also 100 meters).

There are two city maps that you'll be working with: a grid map (`createGridMap`) and a map of Stanford (`createStanfordMap`), which is derived from [Open Street Maps](#). Please note that you do not need to know the Stanford campus in order to answer the questions below! We have also included instructions on how to create your own maps in `README.md`.

Important: On Windows some of the commands will lead to the python process not exiting and hanging. If you experience this behaviour, simply kill the process as the results will not be affected!

1. Grid City

Consider an infinite city consisting of locations (x, y) where x, y are integers. From each location (x, y) , one can go east, west, north, or south. You start at $(0, 0)$ and want to go to (m, n) , where $m, n \geq 0$. We can define the following search problem to capture this:

- $s_{\text{start}} = (0, 0)$
- $\text{Actions}(s) = \{(+1, 0), (-1, 0), (0, +1), (0, -1)\}$
- $\text{Succ}(s, a) = s + a$
- $\text{Cost}((x, y), a) = 1 + \max(x, 0)$ (it is more expensive as x increases)
- $\text{IsEnd}(s) = \mathbf{1}[s = (m, n)]$

(a) [1 point (Written)]

What is the minimum cost of reaching location (m, n) (note that $m, n \geq 0$) starting from location $(0, 0)$ in the above city? Describe one possible path achieving the minimum cost. Is it unique (i.e., are there multiple paths that achieve the minimum cost)?

What we expect: Provide an expression for the minimum cost. In addition, using 1 - 2 sentences describe one possible minimum cost path and state whether it is unique.

The minimum cost is $n + \frac{m(m+1)}{2}$.

Since the cost of any action from state (x, y) is $1 + \max(x, 0) = 1 + x$ for $x \geq 0$, cost depends only on the current x -coordinate. Any detour west or south adds extra steps at positive cost, so the optimal path uses exactly m eastward and n northward steps. The m eastward steps must be taken at $x = 0, 1, \dots, m-1$ in some order, costing $1 + 2 + \dots + m = \frac{m(m+1)}{2}$ in total (unavoidable). Each northward step costs $1 + x$; to minimize, all n north steps should be taken at $x = 0$ (cost 1 each), giving n total.

Minimum cost path: Move north n times from $(0, 0)$ to $(0, n)$, then move east m times to (m, n) .

Uniqueness: The path is unique. Any northward step taken after at least one eastward step would be at $x \geq 1$, costing ≥ 2 instead of 1, which is suboptimal. Therefore all north steps must precede all east steps, giving exactly one optimal path.

(b) [1.50 points (Written)] How will Uniform Cost Search (UCS) behave on this problem? Mark the following as true or false:

- UCS will never terminate because the number of states is infinite.
- UCS will return the minimum cost path and explore only locations between $(0, 0)$ and (m, n) ; that is, (x, y) such that $0 \leq x \leq m$ and $0 \leq y \leq n$.
- UCS will return the minimum cost path and explore only locations whose past costs are less than the minimum cost from $(0, 0)$ to (m, n) .

What we expect: T/F for each subpart.

- False.** Every action has cost ≥ 1 , so UCS will expand and reach the goal without exploring all states.
- False.** UCS will explore paths that go south since the cost will be less when $x < 0$.
- True.** UCS pops states in non-decreasing order of past cost and stops the moment it pops the goal with cost C^* (where C^* is the minimum cost). Every state that was expanded (i.e., had its successors generated) was popped strictly before the goal, so its past cost is $< C^*$.

(c) [1.50 points (Written)] Now consider running UCS on an arbitrary graph. Mark the following as true or false:

- If you add a connection between two locations, the minimum distance cannot go up.

- ii. If you make the cost of an action from some state small enough (possibly negative), that action will show up in the minimum cost path.
- iii. If you increase the cost of each action by 1, the minimum cost path does not change (even though its cost does).

What we expect: T/F for each subpart.

- i. **True.** New edges only add new options. If existing paths are unaffected then the minimum distance will not increase.
- ii. **False.** If the action leads to a state from which the goal is unreachable, no path through that action can reach the goal regardless of its cost.
- iii. **False.** Adding a constant $+1$ to every edge cost penalizes paths with more edges. Consider two paths: one with 1 edge of cost 5 (total 5) and another with 4 edges each of cost 1 (total 4). The 4-edge path is optimal originally. After adding 1 per edge the costs become 6 vs. 8, so the minimum cost path changes.

2. Finding Shortest Paths

We first start out with the problem of finding the shortest path from a *start location* (e.g., the Gates Building) to some end location. In Google Maps, you can only specify a specific end location (e.g., Coupa Cafe at Green Library).

In this problem, we want to give the user the flexibility of specifying *multiple* possible end locations by specifying a set of "tags" (e.g., so you can say that you want to go to *any* place with food versus a specific location like Tressider).

(a) [8 points (Coding)]

Implement `ShortestPathProblem` so that given a `startLocation` and `endTag`, the minimum cost path corresponds to the shortest path from `startLocation` to *any* location that has the `endTag`.

In particular, you need to implement `startState()`, `isEnd(state)`, `successorsAndCosts(state)`.

Recall the separation between search problem (modeling) and search algorithm (inference). You should focus on modeling (defining the `ShortestPathProblem`); the default search algorithm, `UniformCostSearch` (UCS), is implemented for you in `util.py`.

What we expect: An implementation of the `ShortestPathProblem` class.

(b) [1 point (Written)]

Run `python mapUtil.py > readableStanfordMap.txt` to write a (long-ish) file of the possible locations on the Stanford map along with their tags. Each tag is a `[key]=[value]`. Here are some examples of keys:

- **landmark:** Hand-defined landmarks (from `src/data/stanford-landmarks.json`)
- **amenity:** Various amenity types (e.g., "park", "food")
- **parking:** Assorted parking options (e.g., "underground")

Choose a starting location and end tag (perhaps that's relevant to your daily life) and implement `getStanfordShortestPathProblem()` to create a search problem. Then, run `python grader.py 2b-0-helper` to generate `path.json`. Once generated, run `python visualization.py` to visualize it (opens in browser). Try different start locations and end tags. Pick two settings corresponding to the following:

- A start location and end tag that produced new insight into traveling around campus. Describe whether the system was useful.
- A start location and end tag where the minimum cost path found isn't desirable. Is this due to incorrect modeling assumptions? Explain.

You should feel free to add new landmarks and if you are not at Stanford, follow the instructions in the `README.md` to use your own map and landmarks.

What we expect: A screenshot of the visualization of your two solutions as well as i) one or more sentences describing something interesting you've learned about traveling (around campus, or elsewhere) and ii) something incorrect about either the map or modeling assumptions (such a landmark being out of place, etc.).

I created a custom map of Vancouver (where I currently live) and I found the tools for creating maps, paths and search very interesting.

The first image below is the shortest path from one of my favorite restaurants (Dovetail) to my home. This is the shortest path if I am walking, but this path is not accessible by car. The modelling assumes (incorrectly) that all paths are available for all types of travel (car, bicycle, walking, etc). The second image is the shortest path from the Vogue Theater to my home. Again, this is the shortest path if walking (as it goes through a local park), but it would not be a valid path if driving a car.

Both externalities arise from a fundamental modeling mismatch: the route-planning system optimizes myopically for a single user's travel time and treats the road network as static, when in reality all agents interact - routing one driver onto a street raises the cost for every other driver on that street.

One way to reduce this mismatch is to adopt collaborative routing, where the system models aggregate traffic flow and deliberately distributes drivers across multiple routes of equivalent cost, trading individual optimality for social optimality and preventing any single corridor from being overwhelmed.

3. Finding Shortest Paths with Unordered Waypoints

Let's introduce an even more powerful feature: unordered waypoints! In Google Maps, you can specify an ordered sequence of waypoints that a path must go through – for example, going from point A to point X to point Y to point B, where [X, Y] are "waypoints" (such as a gas station or a friend's house).

However, we want to consider the case where the waypoints are *unordered*: X, Y, so that both $A \rightarrow X \rightarrow Y \rightarrow B$ and $A \rightarrow Y \rightarrow X \rightarrow B$ are allowed. Moreover, X, Y, and B are each specified by a tag like in Problem 2 (e.g., `amenity=food`)

This is a neat feature if you think about your day-to-day life; you might be on your way home after a long day, but need to stop by the package center, Tressider to grab a bite of food, and the bookstore to buy some notebooks. Having the ability to get a short, quick path that hits all these stops might be really convenient (rather than searching over the various waypoint orderings yourself).

(a) [11 points (Coding)]

Implement `WaypointsShortestPathProblem` so that given a `startLocation`, set of `waypointTags`, and an `endTag`, the minimum cost path corresponds to the shortest path from `startLocation` to a location with the `endTag`, such that all of `waypointTags` are covered by some location in the path.

Note that a single location can be used to satisfy multiple tags.

Like in Problem 2, you need to implement `startState()`, `isEnd(state)`, and `successorsAndCosts(state)`.

There are many ways to implement this search problem, so you should think carefully about how to design your `State`. We want to optimize for a compact state space so that search is as efficient as possible.

What we expect: An implementation of the `WaypointsShortestPathProblem` class. To get full credit, your implementation must have the right asymptotic dependence on the number of locations and waypoints. Note that your code will timeout if you do this incorrectly.

(b) [1.50 points (Written)]

If there are n locations and k waypoint tags, what is the maximum number of states that UCS could visit?

What we expect: A mathematical expression that depends on n and k , with a brief explanation justifying it.

The maximum number of states UCS could visit is $n \cdot 2^k$.

Justification: Each state in `WaypointsShortestPathProblem` is a pair (location, memory), where:

- **location** can be any of the n locations in the map, and
- **memory** is a frozenset recording which waypoint tags have been covered so far. Since memory is always a subset of the k waypoint tags, there are at most 2^k distinct memory values (one per subset of the k tags).

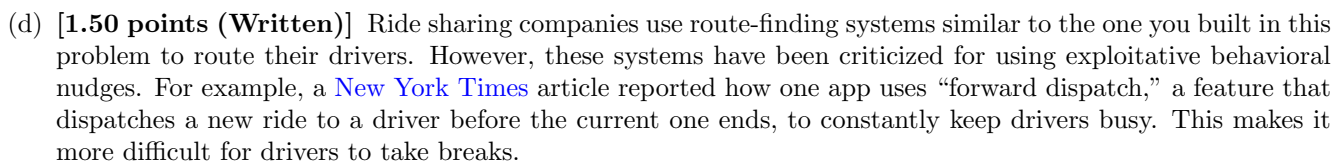
Combining both dimensions, the state space has at most $n \cdot 2^k$ distinct states, so UCS visits at most $n \cdot 2^k$ states.

(c) [2 points (Written)]

Choose a starting location, set of waypoint tags, and an end tag (perhaps that captures an interesting route planning problem relevant to you), and implement `getStanfordWaypointsShortestPathProblem()` to create a search problem. Then, similar to Problem 1b, run `python grader.py 3c-0-helper` to generate `path.json`. Once generated, run `python visualization.py` to visualize it (opens in browser).

What we expect: A screenshot of the visualized path with a list of the waypoint tags you selected. In addition, provide one or more sentences describing unexpected or interesting features of the selected path. Does it match your expectations? What does this path fail to capture that might be important?

I created a custom map of Vancouver (where I currently live) and I found the tools for creating maps, paths and search algorithms very interesting. I added tags: 'landmark=theater', 'amenity=ferry terminal', 'landmark=dovetail' and 'landmark=home'. For the path, I chose the starting point as the 'landmark=dovetail' location, and the endpoint as 'landmark=home'. Waypoints were 'landmark=theater' and 'amenity=ferry terminal'. I chose these locations specifically so that the optimal path would require some backtracking - the start and end locations were near the center, with one waypoint north of startpoint and the other waypoint south of the startpoint. The algorithm worked as expected, it was forced to travel to the closer waypoint first (the southern waypoint), and then backtrack north to the northern waypoint, before having to return to the center to go to the endpoint. Very impressive.



What waypoints could you include from one drop-off location to the next pick-up location to improve the driver experience? What information about drivers would you need to determine appropriate waypoints?

What we expect: Provide 3-5 sentences describing (a) the waypoints you would include to improve the driver experience, (b) at least one dimension of drivers' identity that could inform selection of waypoints, and (c) a potential negative use of the waypoints feature that a company could use to increase profits while harming the driver experience or driver health, or downsides in collecting the appropriate information needed to recommend the waypoints.

A key dimension of driver identity that should inform waypoint selection is shift duration and fatigue level: a driver who has been on the road for four or more hours has different needs than one who just started, and the system could use logged hours to recommend longer breaks or food stops at appropriate intervals, similar to legally mandated rest rules for truck drivers.

However, this same feature could be weaponized by a company to increase profits at drivers' expense: rather than recommending breaks, the company could insert mandatory waypoints through high-demand surge zones between rides, extracting additional unpaid repositioning labor from drivers or routing them past partner businesses (e.g., sponsored gas stations) that generate revenue for the platform.

Furthermore, collecting the personal health and schedule data needed to recommend appropriate rest waypoints raises serious privacy risks—detailed records of a driver's fatigue patterns, dietary habits, and physical location throughout the day could be sold to insurers or advertisers, or used to discriminate against drivers with health conditions, highlighting the dual-use nature of even well-intentioned data collection.

4. Speeding up Search with A*

In this problem, we will explore how to speed up search by reducing the number of states that need to be expanded using various A* heuristics.

(a) [4 points (Coding)]

In this problem, you will implement A* to speed up the search. Recall that in class, A* is just UCS with a different cost function (a new search problem), so we will implement A* via a *reduction* to UCS.

In particular, you should implement `aStarReduction()` which takes a `search problem` and a `heuristic` as input and returns a new search problem.

What we expect: An implementation of the `NewSearchProblem` class in the `aStarReduction(problem, heuristic)` function. As in prior problems, you need to implement `startState()`, `isEnd(state)`, and `successorsAndCosts(state)`.

(b) [6 points (Coding)]

We are now ready to speed up search by simply implementing various heuristics. First, implement `StraightLineHeuristic` for Problem 2, which returns the straight line distance from a state to any location with the end tag. Note: you might want to precompute some things so that evaluating the heuristic is fast.

What we expect: An implementation of the `StraightLineHeuristic` class.

(c) [7 points (Coding)]

Next, implement `NoWaypointsHeuristic` for Problem 3, which returns the minimum cost path from a state to any location with the end tag but ignoring all the waypoints (essentially the solution to Problem 2, so you can reuse that if you'd like). Note: you might want to precompute some things so that evaluating the heuristic is fast. Helpful comments are provided in the code.

What we expect: An implementation of the `NoWaypointsHeuristic` class.

5. AI Tutor

In this problem, you'll participate in an interactive tutoring session with an AI chatbot. The sessions focus on dynamic programming and beam search. By the end, you should be able to explain and apply the basic principles of both techniques, understanding how they help AI systems make efficient and informed decisions.

We recommend using the below prompt template:

Prompt Template

INSERT PROMPT HERE. Keep the session around 15–20 minutes and interactive.

Guidelines

- Adjust difficulty: if I miss 2 in a row → slow + simpler; if I get 2 quickly → a bit harder or add a twist.
- Use tiny, deterministic examples (small numbers, short tables, brief ASCII diagrams, code blocks). Show actual *run-throughs* (step-by-step states) if helps.
- Use Code if it clarifies: ≤ 2 Python/NumPy blocks, each ≤ 20 lines; ideally no imports beyond `import numpy as np`.
- Do NOT solve graded work. If I paste any, refuse and make a similar practice item.
- Be CONCISE. Don't make notation complicated/unnecessary. Focus on intuitions. Code blocks should be short with simple variable names.
- I may interrupt and ask to start somewhere else. Simply adjust and adapt from there.

Suggested flow (flexible; use judgment)

- Start by explaining the topic in a few simple, but technically accurate sentences (explain core idea (≤ 6 sentences): intuition + minimal notation + why it matters + one common pitfall). Use basic notations and visualizations if necessary (CS221 requires basic math/programming backgrounds). For topics about programming, OK to start with concise code blocks.
- Quick check-in (≤ 1 –2 min): ask 2 short questions about prior exposure and specific goal.
- Exercises: use a tiny instance and show intermediate states. Give exercises that build on each other. OK to give multiple exercises at the same time.
 - e.g., for graph search: a 4–5 node weighted graph; show the frontier/priority-queue after each of 2–3 steps and which edges relax.
 - for DP/MDP/RL/Logic/ML, do an equivalently small, concrete step-by-step trace.
 - If code clarifies, include one concise NumPy block.
- 3 quick checks: increasing difficulty; give a hint first; reveal answers only after I try or ask.
- Assessment and feedback: Give me feedback on my performance - what I understood well, what needs improvement, and specific concepts to review further.
- Session reflection: Ask me for feedback on how the tutoring session went - what worked well, what could be improved, and if the pacing/difficulty was appropriate.
- Recap: a 60-second summary

(a) [0.50 points (Written)] Practice Dynamic Programming with an AI tutor!

Use an AI chatbot (e.g., ChatGPT, Claude, Gemini, or [Stanford AI Playground](#)).

For the prompt, use **"Teach me basic Dynamic Programming interactively"**. We recommend using the template provided above.

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be around 15–20 minutes and interactive!

Dynamic Programming chat session:

<https://chatgpt.com/share/69d9d9d0-eb2c-83e8-8e4a-5c48b638eda1>

(b) [0.50 points (Written)] Practice Beam Search with an AI tutor!

Use an AI chatbot (e.g., ChatGPT, Claude, Gemini, or [Stanford AI Playground](#)).

For the prompt, use **"Teach me basic Beam Search interactively"**. We recommend using the template provided above.

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be around 15–20 minutes and interactive!

Beam Search chat session:

<https://chatgpt.com/share/69d9e032-6b00-83e8-85f8-f392daf6d124>

6. Product Ethics Exploration: Documentation, Marketing, and Customer Support

You will choose a modern AI product to analyze throughout the assignments and gain initial access to it.

Suggested resources:

- Pick an AI-powered product or tool (not just a raw LLM chatbot) that is accessible to you. Good candidates include: Cursor, Nano Banana, ElevenLabs, Perplexity, Claude Code, etc. Free trials or student-accessible tools are recommended.
- The product's official website and onboarding/getting started pages
- Privacy policy, Terms of Service, Acceptable Use Policy
- App store listings, GitHub documentation or READMEs (if open-source or developer-targeted)
- Recent news coverage (e.g., TechCrunch, The Verge, Wired, NYT technology section)
- Reddit communities, Hacker News, or user forums for first impressions and experiences

In this part: You will examine how the company communicates with users through marketing, documentation, and support channels.

Why it's important: The gap between marketing promises and actual capabilities, as well as the quality of user support, directly impacts user trust and the product's real-world effectiveness.

Suggested resources:

- Company marketing pages, ads, YouTube demos
- Social media promotions (LinkedIn, TikTok, Instagram, X)
- Reddit, StackOverflow, Discord, or unofficial user communities
- Product FAQ, Help Center
- Customer service emails/chat logs you submit

(a) [0.50 points (Written)] Marketing Channels and Discovery

- How is the product marketed and how do users discover it? Describe the channels and messaging the company uses.

What we expect: In 4-6 sentences, provide a summary of the main marketing channels and discovery avenues for the product, with links, cited examples, and/or screenshots.

Claude Code (by Anthropic) is marketed primarily through developer-native channels: the official product page at anthropic.com/product/claude-code uses taglines such as "Agentic, not autocomplete" and "If you can describe it, you can build it", and major feature launches are announced via the Anthropic News blog and the official GitHub repository (github.com/anthropics/claude-code), which doubles as a distribution and community hub. Organic discovery has been a major driver - Claude Code repeatedly hit number 1 on Hacker News, and five third-party YouTube tutorials appeared within 48 hours of significant feature updates, reflecting strong word-of-mouth among developers.

In September 2025 Anthropic launched its first paid brand campaign, "Keep Thinking", through the agency Mother, running across sports broadcasts, streaming services (Netflix, Hulu), print outlets (New York Times, Wall Street Journal), and out-of-home advertising in over 12 cities (Axios, Sep. 2025).

Claude Code is also actively promoted on X/Twitter via @AnthropicAI and a dedicated @claude/code account, and through the Reddit communities r/ClaudeAI (688K members) and r/ClaudeCode (96K members), where users share workflows and discover the product organically.

(b) [0.50 points (Written)] Marketing Strategy and Mission Alignment

- Summarize what you believe the company's **core mission** is, based on official statements (About page, press releases, etc.).
- Discuss whether the product's marketing strategy and messaging **align or conflict with the company's mission**. In your view, does the marketing sensationalize or misrepresent what the product can do, or is it fair and accurate? Provide rationale and cite examples (ads, taglines, posts, etc.).

What we expect: In 4-6 sentences, clearly articulate whether the company's marketing supports, contradicts, or sensationalizes its stated mission and product capabilities, providing specific evidence.

Anthropic's stated core mission, from anthropic.com/company, is "the responsible development and maintenance of advanced AI for the long-term benefit of humanity", and the company operates as a Public Benefit Corporation with explicit values around safety, integrity, and user care.

Claude Code's marketing is broadly aligned with this mission: the "Keep Thinking" campaign is notably cautious and human-first - celebrating "the joy of not knowing yet" and positioning AI as an amplifier of human cognition rather than a replacement - and the product page explicitly highlights safety features such as permission-based actions and built-in classifiers distinguishing safe from risky operations.

However, there is tension between the marketing promise - "Claude Code handles the rest" - and documented user experience: in early 2026, users and engineers (including an AMD director-level engineer in a widely-covered GitHub issue) reported that the model began ignoring instructions, claiming tasks were complete when they were not, and causing API costs to increase by over 100×, with no formal communication from Anthropic about the regressions.

While the marketing messaging itself is not sensationalized, the gap between the "agentic" capability claims and the delivered product quality during regression periods arguably conflicts with Anthropic's stated value of integrity - suggesting that honest marketing must be paired with equally transparent operational communication when things go wrong.

(c) [0.50 points (Written)] **Feedback, Help, and Customer Support**

- How can users contact the company or get support? Describe the official support channels, any community-based help resources, and your overall impression of how easy it is to get assistance.

What we expect: In 2-3 sentences, summarize the avenues available for support and community help, and comment on the effectiveness of official and unofficial documentation resources.

Official support is available through the Claude Help Center (support.claude.com), which routes users first to an AI chatbot (Fin) and then, for paid subscribers (Pro, Max, Team, Enterprise), to human specialists via email; free-tier users have access only to the bot and documentation, with no live chat or phone support.

The technical documentation at code.claude.com/docs is comprehensive - covering installation, CLI reference, MCP integration, GitHub Actions, and troubleshooting - and the official GitHub repository (github.com/anthropics/claude-code) functions as a high-quality community support channel where Anthropic engineers actively respond to bug reports.

Unofficial resources are exceptionally active: the Anthropic Discord server (82K+ members), r/ClaudeAI (688K members), and r/ClaudeCode (96K members) collectively handle a large volume of real-world troubleshooting and workarounds, often surfacing fixes faster than official channels - making community resources arguably more effective for day-to-day issues than the official support pipeline.

(d) [0.50 points (Written)] **Submitting Customer Support Queries**

- Submit **two customer support queries** based on your personal experience using the product. These should reflect genuine questions or typical issues you encountered. List those queries here and reflect in 1-2 sentences how customer support responded to and handled your queries.

What we expect: Include two well-constructed sample support emails or chat messages, each clearly stating a question, problem, or request for help and, for each query, 1-2 sentences reflecting on how the company handled that query.

Customer Support Request 1 - Abnormal usage limit drain (GitHub Issue #41930, April 2026):

"Single short prompts - including 'hello' - are consuming 2-7% of a full 5-hour session quota. Some users report depleting a 5-hour session in under 20 minutes. Users paying \$20-\$200/month are receiving a fraction of advertised value, with no formal blog post, email, or status page update from Anthropic - only informal engineer tweets. The silence and lack of communication is more damaging to trust than the bug

itself."

Filed by Marcus Puchalla on github.com/anthropics/claude-code/issues/41930. Anthropic acknowledged the issue via engineer responses on GitHub and X but issued no formal service communication; the community-identified root causes (two prompt-caching bugs inflating token counts 10–20×, a session-resume bug generating 652,069 tokens autonomously, and undisclosed peak-hour throttling) were eventually patched. The handling reflected a gap between the company's stated integrity values and its operational transparency during an incident affecting all paid tiers.

Customer Support Request 2 - Model quality regression for complex engineering tasks (GitHub Issue #42796, April 2026):

"Claude Code is unusable for complex engineering tasks with the recent updates. The model now ignores instructions, does the opposite of what is requested, and claims tasks are complete when they are not. My API costs rose from \$345 in February to \$42,121 in March - a 122× increase - for the same workload. Quantitative analysis of 6,852 session files shows the Read:Edit ratio collapsed from 6.6 to 2.0, meaning the model is making changes without adequate research, and stop-hook violations went from zero to 173 in 17 days after a model update."

Filed by Stella Laurenzo (AMD director-level engineer) on github.com/anthropics/claude-code/issues/42796, and covered by *The Register* (April 6, 2026). Anthropic closed the issue as "Completed" after the root cause was traced to the rollout of thinking content redaction (`redact-thinking-2026-02-12`), which reduced extended thinking depth by approximately 67%; the thoroughness of the community's own forensic analysis highlighted the inadequacy of official communication around model changes.

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

The minimum cost is $n + \frac{m(m+1)}{2}$.

Since the cost of any action from state (x, y) is $1 + \max(x, 0) = 1 + x$ for $x \geq 0$, cost depends only on the current x -coordinate. Any detour west or south adds extra steps at positive cost, so the optimal path uses exactly m eastward and n northward steps. The m eastward steps must be taken at $x = 0, 1, \dots, m-1$ in some order, costing $1 + 2 + \dots + m = \frac{m(m+1)}{2}$ in total (unavoidable). Each northward step costs $1 + x$; to minimize, all n north steps should be taken at $x = 0$ (cost 1 each), giving n total.

Minimum cost path: Move north n times from $(0, 0)$ to $(0, n)$, then move east m times to (m, n) .

Uniqueness: The path is unique. Any northward step taken after at least one eastward step would be at $x \geq 1$, costing ≥ 2 instead of 1, which is suboptimal. Therefore all north steps must precede all east steps, giving exactly one optimal path.

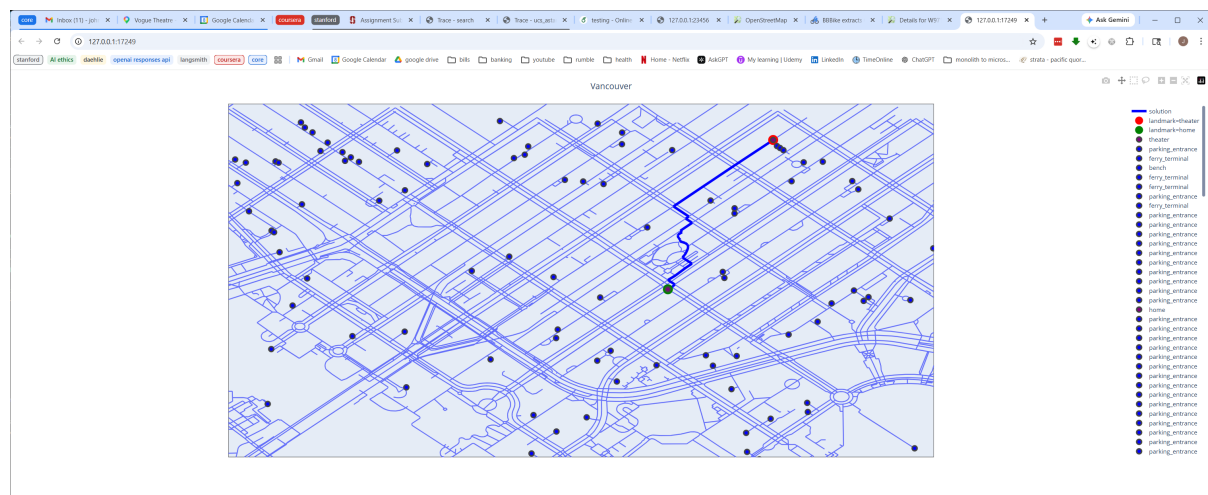
1.b

1. **False.** Every action has cost ≥ 1 , so UCS will expand and reach the goal without exploring all states.
2. **False.** UCS will explore paths that go south since the cost will be less when $x < 0$.
3. **True.** UCS pops states in non-decreasing order of past cost and stops the moment it pops the goal with cost C^* (where C^* is the minimum cost). Every state that was expanded (i.e., had its successors generated) was popped strictly before the goal, so its past cost is $< C^*$.

1.c

1. **True.** New edges only add new options. If existing paths are unaffected then the minimum distance will not increase.
2. **False.** If the action leads to a state from which the goal is unreachable, no path through that action can reach the goal regardless of its cost.
3. **False.** Adding a constant $+1$ to every edge cost penalizes paths with more edges. Consider two paths: one with 1 edge of cost 5 (total 5) and another with 4 edges each of cost 1 (total 4). The 4-edge path is optimal originally. After adding 1 per edge the costs become 6 vs. 8, so the minimum cost path changes.

I created a custom map of Vancouver (where I currently live) and I found the tools for creating maps, paths and search very interesting.

[illegible]

2.c

A key negative externality for users is the “Price of Anarchy” effect: when a large fraction of drivers simultaneously use the same app, they are all funneled onto the same “optimal” route, creating new congestion that makes everyone’s travel time worse than if routing had been coordinated - research shows that at near-100% adoption, system-wide performance actually degrades compared to lower adoption rates.

A serious negative externality for non-users is the displacement of high-volume traffic onto residential side streets (overwhelmed by cut-through traffic) that were never designed to handle it, exposing residents to increased noise, air pollution, and safety hazards without any of the travel-time benefits the app provides.

Both externalities arise from a fundamental modeling mismatch: the route-planning system optimizes myopically for a single user’s travel time and treats the road network as static, when in reality all agents interact - routing one driver onto a street raises the cost for every other driver on that street.

One way to reduce this mismatch is to adopt collaborative routing, where the system models aggregate traffic flow and deliberately distributes drivers across multiple routes of equivalent cost, trading individual optimality for social optimality and preventing any single corridor from being overwhelmed.

3.b

The maximum number of states UCS could visit is $n \cdot 2^k$.

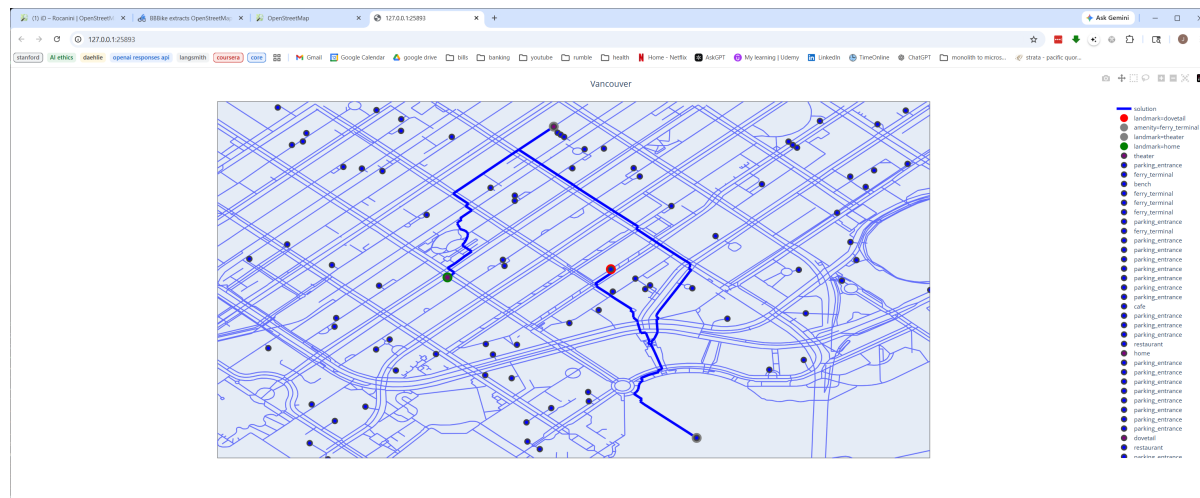
Justification: Each state in `WaypointsShortestPathProblem` is a pair (location, memory), where:

- **location** can be any of the n locations in the map, and
- **memory** is a frozenset recording which waypoint tags have been covered so far. Since memory is always a subset of the k waypoint tags, there are at most 2^k distinct memory values (one per subset of the k tags).

Combining both dimensions, the state space has at most $n \cdot 2^k$ distinct states, so UCS visits at most $n \cdot 2^k$ states.

3.c

I created a custom map of Vancouver (where I currently live) and I found the tools for creating maps, paths and search algorithms very interesting. I added tags: 'landmark=theater', 'amenity=ferry terminal', 'landmark=dovetail' and 'landmark=home'. For the path, I chose the starting point as the 'landmark=dovetail' location, and the endpoint as 'landmark=home'. Waypoints were 'landmark=theater' and 'amenity=ferry terminal'. I chose these locations specifically so that the optimal path would require some backtracking - the start and end locations were near the center, with one waypoint north of startpoint and the other waypoint south of the startpoint. The algorithm worked as expected, it was forced to travel to the closer waypoint first (the southern waypoint), and then backtrack north to the northern waypoint, before having to return to the center to go to the endpoint. Very impressive.



3.d

To improve the driver experience, the unordered waypoints feature could be used to automatically insert rest and recovery stops between a drop-off and the next pick-up-for example, routing through locations tagged `amenity=restaurant`, `amenity=restroom`, or `amenity=gas-station` - giving drivers a genuine break rather than immediately dispatching a new ride via forward dispatch, the technique documented by the New York Times where a new fare is assigned before the current ride ends, psychologically preventing drivers from stopping.

A key dimension of driver identity that should inform waypoint selection is shift duration and fatigue level: a driver who has been on the road for four or more hours has different needs than one who just started, and the system could use logged hours to recommend longer breaks or food stops at appropriate intervals, similar to legally mandated rest rules for truck drivers.

However, this same feature could be weaponized by a company to increase profits at drivers' expense: rather than recommending breaks, the company could insert mandatory waypoints through high-demand surge zones between rides, extracting additional unpaid repositioning labor from drivers or routing them past partner businesses (e.g., sponsored gas stations) that generate revenue for the platform.

Furthermore, collecting the personal health and schedule data needed to recommend appropriate rest waypoints raises serious privacy risks-detailed records of a driver's fatigue patterns, dietary habits, and physical location throughout the day could be sold to insurers or advertisers, or used to discriminate against drivers with health conditions, highlighting the dual-use nature of even well-intentioned data collection.

5.a

Dynamic Programming chat session:

<https://chatgpt.com/share/69d9d9d0-eb2c-83e8-8e4a-5c48b638eda1>

5.b

Beam Search chat session:

<https://chatgpt.com/share/69d9e032-6b00-83e8-85f8-f392daf6d124>

6.a

Claude Code (by Anthropic) is marketed primarily through developer-native channels: the official product page at anthropic.com/product/claude-code uses taglines such as "Agentic, not autocomplete" and "If you can describe it, you can build it", and major feature launches are announced via the Anthropic News blog and the official GitHub repository (github.com/anthropics/claude-code), which doubles as a distribution and community hub. Organic discovery has been a major driver - Claude Code repeatedly hit number 1 on Hacker News, and five third-party YouTube tutorials appeared within 48 hours of significant feature updates, reflecting strong word-of-mouth among developers.

In September 2025 Anthropic launched its first paid brand campaign, "Keep Thinking", through the agency Mother, running across sports broadcasts, streaming services (Netflix, Hulu), print outlets (New York Times, Wall Street Journal), and out-of-home advertising in over 12 cities (Axios, Sep. 2025).

Claude Code is also actively promoted on X/Twitter via @AnthropicAI and a dedicated @claude/code account, and through the Reddit communities r/ClaudeAI (688K members) and r/ClaudeCode (96K members), where users share workflows and discover the product organically.

6.b

Anthropic's stated core mission, from anthropic.com/company, is "the responsible development and maintenance of advanced AI for the long-term benefit of humanity", and the company operates as a Public Benefit Corporation with explicit values around safety, integrity, and user care.

Claude Code's marketing is broadly aligned with this mission: the "Keep Thinking" campaign is notably cautious and human-first - celebrating "the joy of not knowing yet" and positioning AI as an amplifier of human cognition rather than a replacement - and the product page explicitly highlights safety features such as permission-based actions and built-in classifiers distinguishing safe from risky operations.

However, there is tension between the marketing promise - "Claude Code handles the rest" - and documented user experience: in early 2026, users and engineers (including an AMD director-level engineer in a widely-covered GitHub issue) reported that the model began ignoring instructions, claiming tasks were complete when they were not, and causing API costs to increase by over 100×, with no formal communication from Anthropic about the regressions.

While the marketing messaging itself is not sensationalized, the gap between the "agentic" capability claims and the delivered product quality during regression periods arguably conflicts with Anthropic's stated value of integrity - suggesting that honest marketing must be paired with equally transparent operational communication when things go wrong.

6.c

Official support is available through the Claude Help Center (support.claude.com), which routes users first to an AI chatbot (Fin) and then, for paid subscribers (Pro, Max, Team, Enterprise), to human specialists via email; free-tier users have access only to the bot and documentation, with no live chat or phone support.

The technical documentation at code.claude.com/docs is comprehensive - covering installation, CLI reference, MCP integration, GitHub Actions, and troubleshooting - and the official GitHub repository (github.com/anthropics/claude-code) functions as a high-quality community support channel where Anthropic engineers actively respond to bug reports.

Unofficial resources are exceptionally active: the Anthropic Discord server (82K+ members), [r/ClaudeAI](https://www.reddit.com/r/ClaudeAI) (688K members), and [r/ClaudeCode](https://www.reddit.com/r/ClaudeCode) (96K members) collectively handle a large volume of real-world troubleshooting and workarounds, often surfacing fixes faster than official channels - making community resources arguably more effective for day-to-day issues than the official support pipeline.

6.d

Customer Support Request 1 - Abnormal usage limit drain (GitHub Issue #41930, April 2026):

"Single short prompts - including 'hello' - are consuming 2–7% of a full 5-hour session quota. Some users report depleting a 5-hour session in under 20 minutes. Users paying \$20–\$200/month are receiving a fraction of advertised value, with no formal blog post, email, or status page update from Anthropic - only informal engineer tweets. The silence and lack of communication is more damaging to trust than the bug itself."

Filed by Marcus Puchalla on github.com/anthropics/claude-code/issues/41930. Anthropic acknowledged the issue via engineer responses on GitHub and X but issued no formal service communication; the community-identified root causes (two prompt-caching bugs inflating token counts 10–20×, a session-resume bug generating 652,069 tokens autonomously, and undisclosed peak-hour throttling) were eventually patched. The handling reflected a gap between the company's stated integrity values and its operational transparency during an incident affecting all paid tiers.

Customer Support Request 2 - Model quality regression for complex engineering tasks (GitHub Issue #42796, April 2026):

"Claude Code is unusable for complex engineering tasks with the recent updates. The model now ignores instructions, does the opposite of what is requested, and claims tasks are complete when they are not. My API costs rose from \$345 in February to \$42,121 in March - a 122× increase - for the same workload. Quantitative analysis of 6,852 session files shows the Read:Edit ratio collapsed from 6.6 to 2.0, meaning the model is making changes without adequate research, and stop-hook violations went from zero to 173 in 17 days after a model update."

Filed by Stella Laurenzo (AMD director-level engineer) on github.com/anthropics/claude-code/issues/42796, and covered by *The Register* (April 6, 2026). Anthropic closed the issue as "Completed" after the root cause was traced to the rollout of thinking content redaction (`redact-thinking-2026-02-12`), which reduced extended thinking depth by approximately 67%; the thoroughness of the community's own forensic analysis highlighted the inadequacy of official communication around model changes.